COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010	**SECTION:**	CPE21S1
DATE PERFORMED:	July 16, 2026	**DATE SUBMITTED:**	July 20, 2026
NAME:	Carl Andrei D. Torres	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 2.1 **Hands-on Activity 2.1 Arrays, Pointers and Dynamic Memory Allocation**

1. PROCEDURE OUTPUT

INPUT

```
#include <iostream>
#include <string>

class Student{
     private:
        std::string studentName;
        int studentAge;

     public:
     //constructor
     Student(std::string newName ="John Doe", int newAge=18){
          studentName = std::move(newName);
          studentAge = newAge;
          std::cout << "Constructor Called." << std::endl;
};

//Destructor
     ~Student(){
     std::cout <<"Destructor was called" <<std::endl;
     }

//Copy Constructor
     Student (const Student &copyStudent) {
```

```
        std::cout << "Copy Constructor was Called" << std :: endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }
//Display Attributes
    void printDetails(){
        std::cout << this->studentName << " " << this->studentAge <<
std::endl;
    }

};

int main() {
    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;
}
```

OUTPUT

```
PS C:\Users\TIPQC\Documents\CPE010-CPE21S1 (TORRES)\HOA_2.1>  & 'c
e' '--stdout=Microsoft-MIEngine-Out-fqmjj2ct.zrl' '--stderr=Micros
Constructor Called.
Copy Constructor was Called
Constructor Called.
Destructor was called
Destructor was called
Destructor was called
```

Observation	In this program, we use the libraries **<iostream> and <string>** for the printing of the text file. Also, there are **private variables in the class** which are the **std::string studentName and the int studentAge** which are responsible for **Data Organization** of the names and age of the students. Also, there are **public variables that are needed for the display of the outputs which consist of Constructor, Destructor, Copy Constructor, and Display Attributes. Constructor Code which calls for a newName (John Doe) and newAge (18)**

	which outputs the "Constructor Called". Also, **the Destructor Code which deletes the resources which the code consists of (~) that calls "Destructor was Called" at the end of the program itself**. **Copy Constructor**, **copies the studentName and studentAge into a new file name which calls "Copy Constructor was Called"**. **Display Attributes, prints the details of the students**. The **main driver code shows another newName (Roman) and newAge (28) which detects that the new constructor was called which outputs "Constructor Called" after the Copy Constructor Code was executed**.

Table 2-1. Initial Driver Program

INPUT

```
#include <iostream>
#include <string>

class Student{
     private:
        std::string studentName;
        int studentAge;

     public:
     //constructor
     Student(std::string newName ="John Doe", int newAge=18){
          studentName = std::move(newName);
          studentAge = newAge;
          std::cout << "Constructor Called." << std::endl;
};

//Destructor
     ~Student(){
     std::cout <<"Destructor was called" <<std::endl;
     }

//Copy Constructor
     Student (const Student &copyStudent) {
          std::cout << "Copy Constructor was Called" << std :: endl;
          studentName = copyStudent.studentName;
          studentAge = copyStudent.studentAge;
```

```
    }
//Display Attributes
    void printDetails(){
        std::cout << this->studentName << " " << this->studentAge <<
std::endl;
    }

};

int main() {
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack",
"Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

OUTPUT

```
PS C:\Users\TIPQC\Documents\CPE010-CPE21S1 (TORRES)\HOA_2.1>  & 'c:\Users\
o' '--stdout=Microsoft-MIEngine-Out-tw5g0taf.4nt' '--stderr=Microsoft-MIEng
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor was called
Destructor was called
Destructor was called
Destructor was called
Destructor was called
```

Observation	This program is being modified because of the **array size of 5** which shows the **namesList for the names of the 5 students** and **ageList for the age of the 5 students**. It shows that there are **5 outputs in the Constructor Code and Destructor Code each** which is the **same as the array size of 5 students in the driver code itself**.

Table 2-2. Modified Driver Program with Student Lists

LOOP A INPUT

```
for(int i = 0; i < j; i++){ //Loop A
        Student *ptr = new Student (namesList[i], ageList[i]);
        studentList[i] = *ptr;
    }
```

Observation	This loop **prints the names and age of each student in the array of namesList and ageList. It is because of the dereference operator (*ptr) which prints the value not the memory addresses of the names and age at the namesList and ageList**.

LOOP B INPUT

```
for(int i = 0; i < j; i++){ //Loop B
        studentList[i].printDetails();
    }
```

Observation	This loop **calls for the constructor and destructor code which shows the continuous execution and output of the Constructor and Destructor code**.

OUTPUT

```
ode\extensions\ms-vscode.cpptools-1.32.2-win32-x64\debugAdapters\bin\WindowsDebugLaunc
-jfgoz44v.baz' '--pid=Microsoft-MIEngine-Pid-f4db0clq.ch3' '--dbgExe=C:\msys64\ucrt64\
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Carly 15
Freddy 16
Sam 18
Zack 19
Cody 16
Destructor was called
Destructor was called
Destructor was called
Destructor was called
Destructor was called
```

Observation	In the functions inside the class, it **keeps on executing because of the code in Loop B which constantly prints the chain of Constructor and Destructor codes**.

Table 2-3. Final Driver Program

Modifications	Just put the name of the constructor, destructor, and copy constructor with the statement of **"was called for" to show the student name that the constructor, destructor, and copy constructor was called for**. Also, I've used the **Student *ptr = new student (nameList[i], ageList [i] to construct a HEAP memory which calls for each student name and age**. Also, the **delete *ptr was used to delete the temporary heap memory which prints like this ("Destructor was called for Freddy")**.
Observation	I observed that the code in the Main Function has a bug which is the **(Student studentList[j] = {};)** which automatically **executes a 5 default "John Doe" dummy not the nameList one which prints the output of "Constructor was Called" which is not defined which student in the nameList one**. I've modified it by **adding the statement "Constructor called for " and the studentName which prints the output ("Constructor called for Carly")**. Moreover, I've used the **delete *ptr to delete the temporary heap memory so that output will be printed in the compiler**.

Table 2-4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

CODES:

```
#include <iomanip>
#include <iostream>
#include <string>
```

GLOBAL DECLARATION SECTION

CODE ANALYSIS	I've used the **iomanip for the formatting of the output streams for std::cout like for example (std::setw(12) which sets the width in the printing field to align the columns cleanly to output the item names, prices, and total prices**. Also, **usual iostream for the std::cout and std::endl**. Lastly, **string is used for declaring text variables such as name and description and performing string comparisons like getname() == "LETTUCE"**.

```
protected:
    std::string name;
    double price;
    int quantity;
    std::string description;
```

PROTECTED VARIABLES (class Items)

CODE ANALYSIS	I've used **protected variables instead of private because it is accessible inside the base class (Items) and also inside of any derived child classes (Fruit and Vegetable), but it is not accessible from outside the code like the main()**. Also, **Protected Variable allows the derived child classes to modify their attributes like name, price, or description while still hiding these variables from the outside modification in the main()**. The **variable (std::string name) stores the name of the items**. The **variable (double price) stores the unit price of the items which has double that shows the precise decimal for the prices like with extra cents**. The **variable (int quantity) shows the number of items and of course needs the exact number so it's in Integer data type**. Lastly, The **variable (std::string description) stores the category of the item ("Fruit" or "Vegetable").**

```
public:
    //Constructor
    Items(std ::string itemName = "", double itemPrice = 0, int itemQuantity = 0, std::string itemDescription = "Item" )
        : name(itemName), price(itemPrice), quantity(itemQuantity), description(itemDescription) {
        std :: cout << "[Constructor was called] Created " << description << ": " << (name.empty() ? "Empty" : name) << std :: endl;
    }
    //Destructor
    virtual ~Items() {
        std::cout << "[Destructor was called] Destroyed " << description << ": "  << (name.empty() ? "Empty Item" : name) << std :: endl;
    }
    //Copy Constructor
    Items(const Items& other) {
        name = other.name;
        price = other.price;
        quantity = other.quantity;
        description = other.description;
        std::cout << "[Copy Constructor was called] Copied " << name << std :: endl;
    }
    //Copy Assignment Operator
    Items& operator=(const Items& other) {
        if (this != &other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
            description = other.description;
            std::cout << "[Copy Assignment Operator was called] Assigned " << name << std :: endl;
        }
        return *this;
    }
    //calculaterPrice Function
    double calculatePrice() const {
        return price * quantity;
    }
```

```
    //printDetails function
    virtual void printDetails() const {
        std::cout << std :: left << std::setw(12) << description
                  << std :: setw(12) << name
                  << "PHP " << std::setw(8) << price
                  << "x" << std::setw(5) << quantity
                  << "Total: PHP " << calculatePrice() << std :: endl;
    }

    //getName function
    std :: string getName() const { return name;}
};
```

PUBLIC VARIABLES (class Items)

CODE ANALYSIS	In the Public Variables, there are **Constructor, Destructor, Copy Constructor, and Copy Assignment Operator**: ● **Constructor** which **constructs new Items object which initialized at (Items(std::string itemName = "", double itemPrice = 0, int itemQuantity = 0, std::string itemDescription = "Item") : name(itemName), price(itemPrice), quantity(itemQuantity), description(itemDescription) { ... })** which also has **member initializer list like name(itemName) to directly initialize the member variables before the constructor code executes completely**. Also, it has **“” and 0 which sets for default arguments that allows a single argument for each member**. ● **Destructor**, **deletes the object when it’s out of the scope** and I’ve used **Virtual because it guarantees the deletion of the derived objects (Fruit or Vegetable) inside the Items* pointers which allow the derived destructor come first before the base destructor**. ● **Copy Constructor**, initializes a **new Items as a duplicate for the existing Items object and the const reference operator prevents the Copy Constructor on unnecessary memory copies or infinite copies during object passing execution**. ● **Copy Assignment Operator**, it **copies the value from an existing Items object into an already-existing object which I’ve used the (if (this != &other) {...} to prevent an object to accidentally copying itself** and **(return *this) to return the reference to the current object which makes an organized assignment of variables**. ● **(calculatePrice) function**, shows the **arithmetic operation (multiply or *) which is price * quantity** and **I’ve used double to give the exact unit price** and **const to prevent this function to modify any member variables in the class**. ● **(printDetails) function**, shows the **Virtual again to allow the child classes (Fruit and Vegetable) to stop executing if needed a custom output format** and **Const to ensure the printing of the details are correct**. Also, I’ve used **(std::setw) to properly print the attributes of the items in a tabular column**. ● **(getName) function**, I used it to have **read-only access to the protected member “name” from outside of the class which is used in the main() to check getName() == “LETTUCE”**.

```cpp
//Problem 1

class Fruit : public Items {
public:
     Fruit(std::string name = "", double price = 0.0, int quantity = 0)
     : Items(name, price, quantity, "Fruit"){}
};

class Vegetable : public Items {
public:
     Vegetable(std::string name = "", double price = 0.0, int quantity = 0)
     : Items(name, price, quantity, "Vegetable"){}
};
```

PROBLEM 1

CODE ANALYSIS	In this code, there are **two Child Classes** which are **initialized as (class … : public Items {...}) which the Fruit and Vegetable (child classes) inherits from the Items (parent) class** and **the public inheritance was initialized for the accessibility of the Public and Protected Variables like (name, price, quantity, description) and Methods (calculatePrice(), printDetails(), getName())**. It also have a **Constructor Initializer of (Fruit(std::string name = "", double price = 0.0, int quantity = 0) : Items(name, price, quantity, "Fruit"){})** which the **derived class constructor (Items(name, price, quantity, "Fruit"){}) has the base class constructor which is Items(...) and the "Fruit" is in the 4th argument (itemDescription) to Items**. The **default arguments are still in here so that it can modify it like (Fruit("APPLE", 10.0, 7))**. The **Destructor was still called here because the compiler generates a default destructor for "Fruit" that automatically calls ~Items(). Since ~Items() is Virtual, the deletion in Virtual works well**. Lastly, **Copy Constructor & Assignment Operator are called because the compiler automatically generates copies again in the memory so it calls the Items's copy constructor and assignment operator inside**.

```
//Problem 2

size_t listSize = 4;
Items** groceryList = new Items* [listSize];

groceryList[0] = new Fruit("APPLE", 10.0, 7);
groceryList[1] = new Fruit("BANANA", 10.0,8);
groceryList[2] = new Vegetable("BROCCOLI", 60.0,12);
groceryList[3] = new Vegetable("LETTUCE", 50.0, 10);

std :: cout << "\n--- JENNA'S GROCERY LIST DETAILS ---\n";
for (size_t i = 0; i < listSize; i++) {
    if (groceryList[i] != nullptr) {
        groceryList[i]->printDetails();
    }
}
```

PROBLEM 2

CODE ANALYSIS	In this code, I've used the **(size_t listSize) to declare the size of an array which is composed of 4 items**. Also, I've used the **Items** groceryList which declares a pointer to another pointer like Items* means that the pointer points to an array of Items object and Items** means that the pointer points to an array of Items* pointers**. I've **added newItems* also which allows for 4 empty pointers on the heap memory**. Moreover, I've **inputted the objects polymorphically which has "new" that calls a dynamic object on the heap and returns back to its memory address**. I've **created a Output Header which shows Jenna's Grocery List Details and the code underneath it is the loop through 0 to 3 using the size_t i**. Also, I've **used the if (groceryList[i] != nullptr) because it is important for the verification that a valid object exist at pointer slot [i] before executing the methods**. The **Arrow Operator is used to call that the groceryList[i] is an Items* pointer that would print details on the printDetails() function on each object row by row**.

```
//Problem 3

double TotalPrice(Items** list, size_t size) {
     double grandTotal = 0.0;
     for (size_t i =0; i < size; i++) {
          if (list[i] != nullptr) {
               grandTotal += list[i]->calculatePrice();
          }
     }
     return grandTotal;
}
```

```
//Problem 3

std :: cout << "GRAND TOTAL SUM: PHP " << TotalPrice(groceryList, listSize) << "\n";
```

PROBLEM 3

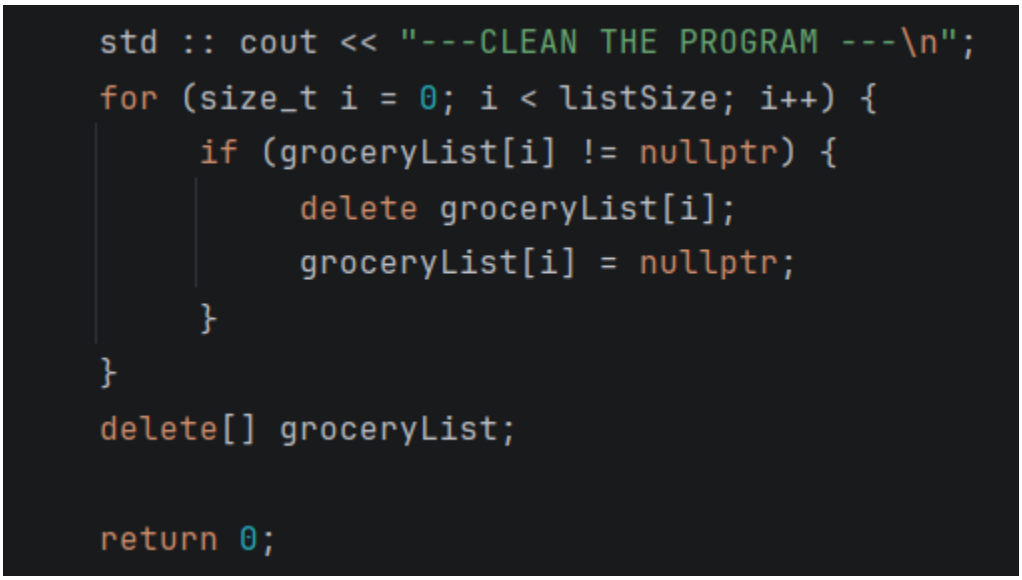
CODE ANALYSIS	In this code, I've **used the (double TotalPrice(Items** list, size_t size) {) which has double for the precision of the unit prices**. Also, **Items** list acquires the address from the dynamic array of the Items* pointers (groceryList) and size_t size that receives the size of the array which is 4 elements**. **(double grandTotal) initializes a starting variable of 0.0. (for (size_t i = 0; i < size; i++) {) initialized a loop from 0 to 3 (0, 1, 2, 3). (if (list[i] != nullptr)) initializes if a valid object pointer exist at [i] so if an item is nullptr**, it **results to false and skips it**. **(list[i]->calculatePrice()) uses an arrow operator to call the calculatePrice() function on the object that are stored on slot [i]. (grandTotal +=) adds the return price (price * quantity) of individual item to the grandTotal**. **(return grandTotal) returns the final value to the main() caller**. Lastly, **(std::cout << "GRAND TOTAL SUM: PHP " << TotalPrice(groceryList, listSize) << "\n";) shows that it passes to the dynamic pointer array (groceryList)** and **size (listSize) to TotalPrice() which immediately outputs the result returned by TotalPrice() by the statement of "GRAND TOTAL SUM: PHP"**.

```
//Problem 4

std :: cout << "\n--- REMOVING LETTUCE AND DEALLOCATE MEMORY ---\n";
for (size_t i = 0; i < listSize; i++) {
     if (groceryList[i] != nullptr && groceryList[i]->getName() == "LETTUCE") {
          delete groceryList[i];
          groceryList[i] = nullptr;
          std :: cout << "SUCCESSFULLY REMOVE LETTUCE FROM THE LIST.\n";
     }
}
std :: cout << "\n--- RE-CALCULATING THE GRAND TOTAL AFTER REMOVING LETTUCE ---\n";
std :: cout << "NEW TOTAL SUM: PHP " << TotalPrice(groceryList, listSize) << "\n";
```

PROBLEM 4

CODE ANALYSIS	In this code, **(for (size_t i = 0; i < size; i++) {) shows the loop from 0 - 3 in the groceryList array. (if (groceryList[i] != nullptr && groceryList[i]->getName() == "LETTUCE") {) shows the (groceryList[i] != nullptr) checks if the slot is not empty** and **groceryList[i]->getName() == "LETTUCE" calls getname() on the object and calls == "LETTUCE" to find the object that matches it. (delete groceryList[i]) calls the destructor function for the groceryList[3] was being removed from the memory itself** and **(groceryList[i] = nullptr) resets the slot 3 into nullptr. Now, (std::cout << "NEW TOTAL SUM: PHP " << TotalPrice(groceryList, listSize) << "\n";) calls again to re-calculate the price of the new items and prints the output.**



```
std :: cout << "---CLEAN THE PROGRAM ---\n";
for (size_t i = 0; i < listSize; i++) {
    if (groceryList[i] != nullptr) {
        delete groceryList[i];
        groceryList[i] = nullptr;
    }
}
delete[] groceryList;

return 0;
```

PROGRAM CLEANUP

CODE ANALYSIS	I've added a program cleanup code because it is needed to **deallocate memory from using destructor functions**. So, I've used the **(for (size_t i = 0; i < listSize; i++) { if (groceryList[i] != nullptr) { delete groceryList[i];groceryList[i] = nullptr; } }) to delete individual heap pointers in the program**. **(delete[] groceryList;) deallocates the dynamic memory of the array pointers itself**. Lastly, **(return 0) calls the main() function to end executing**.

```
--- CREATING GROCERY ITEMS ---
[Constructor was called] Created Fruit: APPLE
[Constructor was called] Created Fruit: BANANA
[Constructor was called] Created Vegetable: BROCCOLI
[Constructor was called] Created Vegetable: LETTUCE

--- JENNA'S GROCERY LIST DETAILS ---
Fruit       APPLE       PHP 10      x7    Total: PHP 70
Fruit       BANANA      PHP 10      x8    Total: PHP 80
Vegetable   BROCCOLI    PHP 60      x12   Total: PHP 720
Vegetable   LETTUCE     PHP 50      x10   Total: PHP 500
GRAND TOTAL SUM: PHP 1370

--- REMOVING LETTUCE AND DEALLOCATE MEMORY ---
[Destructor was called] Destroyed Vegetable: LETTUCE
SUCCESSFULLY REMOVE LETTUCE FROM THE LIST.

--- RE-CALCULATING THE GRAND TOTAL AFTER REMOVING LETTUCE ---
NEW TOTAL SUM: PHP 870
---CLEAN THE PROGRAM ---
[Destructor was called] Destroyed Fruit: APPLE
[Destructor was called] Destroyed Fruit: BANANA
[Destructor was called] Destroyed Vegetable: BROCCOLI

Process finished with exit code 0
```

OUTPUT

Summary of Lessons Learned	In the procedure, I've learned how to create a **Class which contains in the Public Variables are Constructor, Destructor, and Class Constructor**. Constructor Code which **calls for a newName (John Doe) and newAge (18) which outputs the "Constructor Called"**. Destructor Code **which deletes the resources which the code consists of (~) that calls "Destructor was Called" at the end of the program itself**. Copy Constructor, **copies the studentName and studentAge into a new file name which calls "Copy Constructor was Called"**. I've learned also **how to print details by the use of void which loops the execution of the function based on its main driver code**. Also, I've learned **how to create an array which has a syntax of (size_t j = …) for assigning the array size**. I've learned **how to use loop too which has a syntax of for(int i = 0; i < j; i++){) which loops from 0 - j**. In the Supplementary, I've learned **how to use the IOMANIP library to properly organized the variables in a tabular column format using the syntax (std :: setw())**. I've learned that **I need to create another class which is a child class inside the parent class to input the description (Fruits and Vegetables)**. I've also **learned a new keyword Virtual for Polymorphism so that I still can execute the function in the public variables when the code is executing inside the child classes**. In deleting the "LETTUCE" from the supplementary, I've learned how to use the **(if (groceryList[i] != nullptr && groceryList[i]->getName() == "LETTUCE") {) shows the (groceryList[i] != nullptr) checks if the slot is not empty** and **groceryList[i]->getName() == "LETTUCE" calls getname() on the object and calls == "LETTUCE" to find the object that matches it.** I've learned also **how to clean the program at the end** because **it is important to delete all the variables inside of a memory at the end** to prevent **errors in running Destructors properly.**
Analysis of the Procedure	In the procedure, There is a **Class which contains in the Public Variables are Constructor, Destructor, and Class Constructor**. Constructor Code which **calls for a newName (John Doe) and newAge (18) which outputs the "Constructor Called"**. Destructor Code **which deletes the resources which the code consists of (~) that calls "Destructor was Called" at the end of the program itself**. Copy Constructor, **copies the studentName and studentAge into a new file name which calls "Copy Constructor was Called"**. **By the use of void, you can print the details which loops the execution of the function based on its main driver code**. Also, we've **created an array which has a syntax of (size_t j = 5) for assigning the array size of 5**. We use **loop A which has a syntax of for(int i = 0; i < j; i++){) which loops from 0 - 4 and outputs the names and age of each student in the array of namesList and ageList**. It is **because of the dereference operator (*ptr) which prints the value not the memory addresses of the names and age at the namesList and ageList**. In the **loop B**, **calls for the constructor and destructor code which shows the continuous execution and output of the Constructor and Destructor code based on the list size**.

Analysis of the Supplementary Activity	In the Supplementary, The program **uses a base class called Items**. This **base class has protected members for name, price, quantity, and description**. It also **has public functions like a constructor (with and without parameters), a virtual destructor, functions for copying (copy constructor and copy assignment operator), a way to calculate the total price, a virtual function to print details in a tabular column format, and a get name function**. **Two classes, Fruit and Vegetable**, are **derived from Items**. They **inherit publicly and use constructor initializer to send information up to the Items base class, along with a special label for their type**. There's also a **separate global function called TotalPrice**. This function **takes a pointer to a pointer of Items and a size**. It **adds up the costs, making sure to handle cases where pointers might be null**. The main() function runs the program step by step. First, **it allocates memory on the heap for an array of pointers that will hold different kinds of Items objects**. Then, **it goes through the objects, displays their details in a formatted way, searches for a specific item, and deletes it, setting the pointer to null afterward**. It **recalculates the total price and finally cleans up by deleting any remaining objects on the heap**, and then **freeing the memory for the pointer array itself to prevent errors on running Destructors.**.
Conclusions or Feedback	In Conclusion, this program shows **how to handle a flexible array that can hold different kinds of data or variables**. It uses **allocated memory that can grow as needed**. This means **it's grows very fast, taking constant time to get or change items if you know their position in the array so it can easily use this program to operate correctly**. Finding **specific items, like "LETTUCE," or adding up numbers takes time proportional to the number of items, which is as efficient as possible for these tasks when you properly execute this program**. This program **also manages how objects are created and destroyed correctly by the use of Constructor, Destructor, Copy Constructor, and Copy Assignment Operator**. It uses **features like virtual functions and carefully handles different pointers to prevent memory errors and make sure everything runs smoothly, even when the list or array changes size**. I think I need to improve on **how to utilize new keyword such as Virtual efficiently on my code because it is very useful for preventing memory errors from the program from executing different functions enabling its Polymorphism on the program**. Also, I need to **improve my C++ knowledge on coding classes because its like I'm coming back from Programming Logic and Design days** because of the C++ language so I want to keep learning C++ Language again.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Assistance and Explanation of the new C++ data allocation concepts for Code Analysis.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Usage on how the code works per function (Code Analysis) and also the new concepts in C++ data allocation like (Virtual) for polymorphism which makes the code executes efficiently.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.